Week 3

1. Which statement best explains Spark's fault tolerance mechanism?

A. Every RDD is replicated across all nodes.
B. Every transformation result is written to HDFS.
C. Spark tracks lineage and recomputes lost partitions.
D. Spark checkpoints every operation by default.

Answer: C

Explanation

A. Every RDD is replicated across all nodes (Incorrect): Spark does not replicate every RDD by default; fault tolerance is mainly achieved through lineage.

B. Every transformation result is written to HDFS (Incorrect): Writing every intermediate result to HDFS would be expensive and is not Spark's default recovery mechanism.

C. Spark tracks lineage and recomputes lost partitions (Correct): Spark maintains lineage information and rebuilds only the lost partitions when a failure occurs.

D. Spark checkpoints every operation by default (Incorrect): Checkpointing is optional and must be explicitly enabled; Spark does not checkpoint every operation automatically.

2. Which operation below is an Action rather than a Transformation?

A. collect()
B. join()
C. map()
D. filter()

Answer: A

Explanation

A. collect() (Correct): `collect()` is an Action that triggers execution and returns all elements of the RDD to the driver program.

B. join() (Incorrect): `join()` is a Transformation that combines two RDDs based on keys and creates a new RDD.

C. map() (Incorrect): `map()` is a Transformation that applies a function to each element and produces a new RDD.

D. filter() (Incorrect): `filter()` is a Transformation that selects elements satisfying a condition and returns a new RDD.

3. **Which Spark component is responsible for splitting a DAG into stages?**

A. SparkContext
B. TaskScheduler
C. DAGScheduler
D. BlockManager

Answer: C

Explanation

A. SparkContext (Incorrect): SparkContext is the entry point of a Spark application and does not split DAGs into stages.

B. TaskScheduler (Incorrect): TaskScheduler launches tasks on worker nodes after stages have already been created by the DAGScheduler.

C. DAGScheduler (Correct): DAGScheduler analyzes the DAG of operations and divides it into stages based on shuffle boundaries before execution.

D. BlockManager (Incorrect): BlockManager manages cached data blocks and storage, not DAG stage creation.

4. **Why is `reduceByKey()` generally preferred over `groupByKey()` for aggregation?**

A. It automatically sorts keys.
B. It uses combiners on the map side.
C. It creates fewer partitions.
D. It avoids shuffling completely.

Answer: B

Explanation

A. It automatically sorts keys (Incorrect): `reduceByKey()` aggregates values but does not automatically sort keys.

B. It uses combiners on the map side (Correct): `reduceByKey()` performs local aggregation before shuffling data across the network, reducing data transfer and improving performance.

C. It creates fewer partitions (Incorrect): The number of partitions depends on configuration and is not the main reason for preferring `reduceByKey()`.

D. It avoids shuffling completely (Incorrect): `reduceByKey()` still performs a shuffle, but the amount of shuffled data is reduced through local aggregation.

5. **In Spark, which operation triggers actual execution of a DAG?**

A. map()
B. filter()
C. join()
D. count()

Answer: D

Explanation

A. map() (Incorrect): `map()` is a Transformation that only creates a new RDD and does not execute immediately.

B. filter() (Incorrect): `filter()` is a Transformation that lazily defines a filtering operation without triggering execution.

C. join() (Incorrect): `join()` is a Transformation that combines RDDs but waits for an Action before execution begins.

D. count() (Correct): `count()` is an Action that triggers the execution of the DAG and returns the number of elements in the RDD.

6. **Which statement about RDDs is FALSE?**

A. RDDs are immutable.
B. RDDs can be cached for reuse.

C. RDDs automatically rebuild lost data using lineage.
D. RDDs support direct modification of existing partitions.

Answer: D

Explanation

A. RDDs are immutable (True): Once created, an RDD cannot be changed; transformations create new RDDs.

B. RDDs can be cached for reuse (True): Spark allows RDDs to be cached in memory for faster access in iterative computations.

C. RDDs automatically rebuild lost data using lineage (True): Spark uses lineage information to recompute lost partitions when failures occur.

D. RDDs support direct modification of existing partitions (Correct - FALSE Statement): RDDs are read-only (immutable) and cannot be modified after creation.

7. **Which feature helps Spark recover lost RDD partitions?**

A. Replication
B. Lineage
C. Compression
D. Scheduling

Answer: B

Explanation

A. Replication (Incorrect): Replication stores multiple copies of data for fault tolerance, but RDD recovery in Spark primarily relies on lineage, not replication.

B. Lineage (Correct): Spark tracks the sequence of transformations applied to an RDD and can recompute lost partitions when failures occur.

C. Compression (Incorrect): Compression reduces storage or network usage but does not help recover lost partitions.

D. Scheduling (Incorrect): Scheduling manages task execution and resource allocation, not data recovery.

8. **SparkContext is:**

A. Storage system
B. Main entry point to Spark functionality
C. Worker node
D. Scheduler

Answer: B

Explanation

A. Storage system (Incorrect): SparkContext does not store data; storage is handled by systems like HDFS or Spark's storage layer.

B. Main entry point to Spark functionality (Correct): SparkContext is the entry point of a Spark application and is used to create RDDs and access Spark services.

C. Worker node (Incorrect): A worker node executes tasks assigned by the cluster manager, whereas SparkContext runs on the driver.

D. Scheduler (Incorrect): Scheduling is handled by components such as DAGScheduler and TaskScheduler, not SparkContext.

9. **Which Spark component launches tasks on worker nodes and retries failed tasks?**

A. SparkContext
B. DAGScheduler
C. BlockManager
D. TaskScheduler

Answer: D

Explanation

A. SparkContext (Incorrect): SparkContext is the entry point of a Spark application and does not directly schedule tasks.

B. DAGScheduler (Incorrect): DAGScheduler divides the DAG into stages and submits them, but it does not directly launch tasks on workers.

C. BlockManager (Incorrect): BlockManager manages storage, caching, and retrieval of data blocks, not task execution.

D. TaskScheduler (Correct): TaskScheduler is responsible for launching tasks on worker nodes and retrying failed or slow (straggler) tasks.

10. **A Spark application performs multiple iterations over the same dataset. Which feature provides the greatest performance improvement?**

A. HDFS replication
B. Data compression
C. Caching/Persistence of RDDs
D. Increasing block size

Answer: C

Explanation

A. HDFS replication (Incorrect): Replication improves fault tolerance and data availability but does not significantly speed up iterative computations.

B. Data compression (Incorrect): Compression reduces storage and network usage but does not eliminate repeated data access and computation overhead.

C. Caching/Persistence of RDDs (Correct): Caching stores RDDs in memory, allowing repeated iterations to reuse data without re-reading or recomputing it, significantly improving performance.

D. Increasing block size (Incorrect): Larger block sizes may affect storage efficiency but do not provide the major performance gains achieved through in-memory caching for iterative workloads.